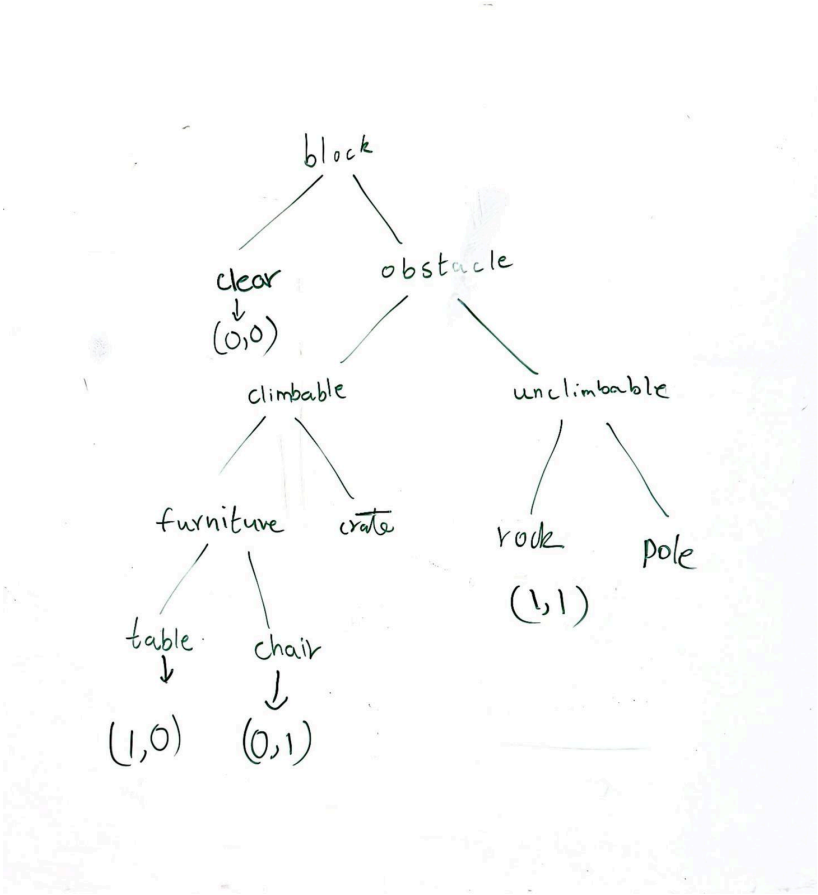
Joseph Coombs, Muhammad Abdullah, Abdul Rehman
Problem #8: Semantic Networks & Planning

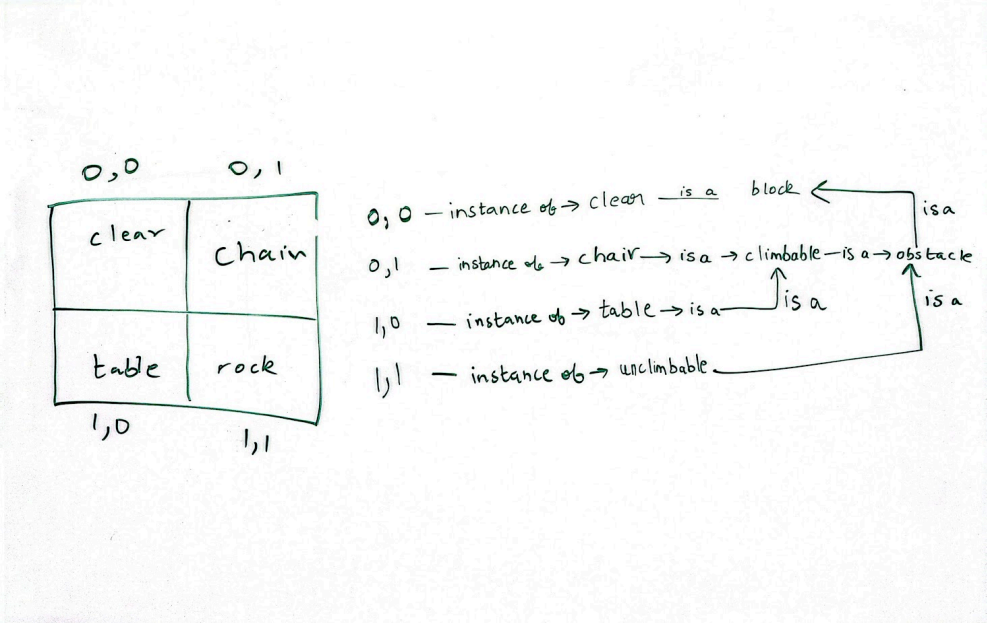
Semantic Networks:

The images below help show a hierarchy (with block at the top) that shows how we can reason with certain categories of objects rather than writing individual situations or scenarios. For example, each block can either be clear or an obstacle, an obstacle is either climbable or unclimbable, and so on. We have 2 unclimbable obstacles here: rock and pole. Climbable obstacles include crates or furniture items, and with that, we have 2 furniture items: tables and chairs, which are climbable obstacles.



The second image shows this hierarchy in action. Given the 2x2 grid below, (0,0) represents a clear space, (0,1) represents a chair, (1, 0) represents a table, and (1,1) represents a rock. (0,0) simply uses an “isa” rule to show that it is a block since it is clear of any obstacles of any kind. (0,1) is an instance of a chair, which given our hierarchy, can show that it is climbable, meaning that it “isa” obstacle. (1,0) is an instance of a table, another example of climbable furniture in our environment. As a result, we can connect it to an “isa” climbable function which then goes along the same hierarchy as our other pieces of furniture. Finally, since (1,1) is a rock, it is clearly unclimbable and therefore an obstacle in our diagram.

Joseph Coombs, Muhammad Abdullah, Abdul Rehman
Problem #8: Semantic Networks & Planning

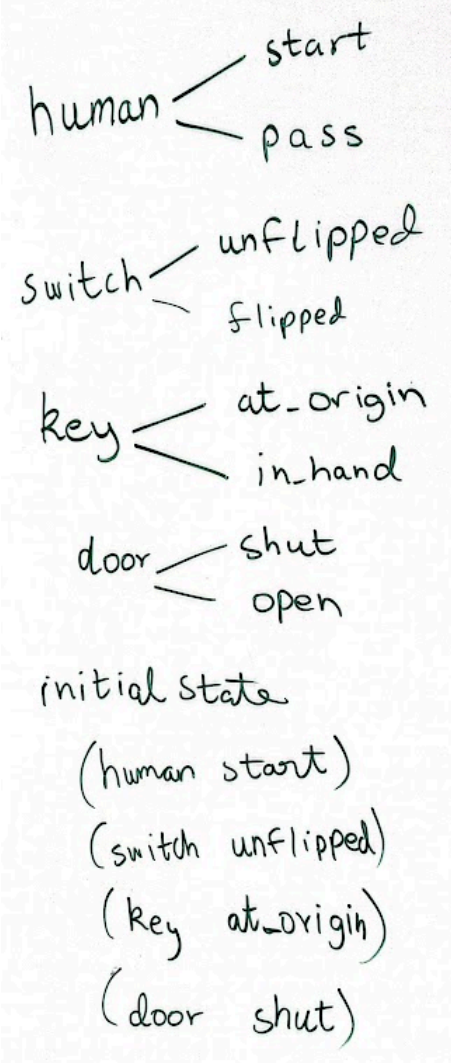


These connections between the different functions for each instance help expand our ability to reason, making said reasoning much more dynamic and adaptable to our given environment.

Joseph Coombs, Muhammad Abdullah, Abdul Rehman
Problem #8: Semantic Networks & Planning

Planning:

In order to pass through, the switch must be unflipped, key must be in hand to open the lock and the door must be open. This initial state is noted above in the aforementioned photo. First, we need to prioritize flipping the switch before simply grabbing the key. We can do this by checking if we are in the initial state (aka, no progress has been made toward reaching the goal) and if so, then flip the switch. Note how the key can't be grabbed if the key is still at the origin and the switch is still unflipped. This is done by first checking the original state with the current state, and, as aforementioned, flipping the switch rather than simply running to the key. After that, if the key is still at its origin, we already know that the switch is flipped and the key can be grabbed, so we add the key to our hand. Finally, we check if the door is still shut and if the key is in our hand. And if that is satisfied, then we have won, and we just need to open the door and reach the goal.



Joseph Coombs, Muhammad Abdullah, Abdul Rehman
Problem #8: Semantic Networks & Planning

```
Original state:
(human start)
(switch unflipped)
(key at_origin)
(door shut)
```

```
If (switch unflipped) and (key at_origin)
Then
delete (switch unflipped)
Add (switch flipped)

If (key at_origin)
Then
Delete (key at_origin)
Add (key in_hand)

If (door shut) (key in_hand) (switch flipped)
Then
Delete (door shut)
Add (door open)

If (door open)
Then
(human pass)
```